Production-Grade Guide

The complete technical reference for the Agentic Research Agent after its production hardening: architecture and diagrams, full configuration, step-by-step local run, server deployment (Docker · Compose · Kubernetes), and how to scale it on real infrastructure.

Document type: Technical & Deployment Guide | Generated: 2026-06-05 | Version: 0.1.0

Runtime: Python 3.13 | 64 tests · ruff · mypy clean

Contents

1. What's New (Production Hardening)

The agent began as a CLI/library package. It now also ships as a horizontally-scalable HTTP service with the operational controls a real deployment needs. Everything below is implemented and tested — not aspirational.

FastAPI service

Async API with `/v1/ask`, `/v1/stream` (SSE), built once at startup and shared across requests.

Durable memory

Pluggable checkpointer: `memory` / `sqlite` / `postgres` — survives restarts and is shareable across replicas.

6 LLM providers

Groq, OpenAI, Azure OpenAI, Anthropic, Gemini, Ollama — chosen by env var, with per-provider default models.

Auth & rate limiting

API-key auth (`X-API-Key`) and a per-identity sliding-window limiter.

Health & metrics

`/health/live`, `/health/ready`, Prometheus `/metrics`; request IDs and a hard request timeout.

Resilience

Provider retries, LLM timeouts, and recovery/resampling for provider tool-call hiccups.

Observability

Structured JSON logs in production, run IDs & durations, optional LangSmith tracing.

Containers & CI

Multi-stage Dockerfile (non-root, healthcheck), docker-compose (API + Postgres), GitHub Actions pipeline.

Content normalization

Clean text from providers that return content blocks (Gemini/Anthropic), not raw dicts.

2. Architecture & Diagrams

Stateless API replicas hold no session state; conversation memory, vectors, and metrics live in shared/observable backends. Knowledge-base ingestion runs out-of-band.

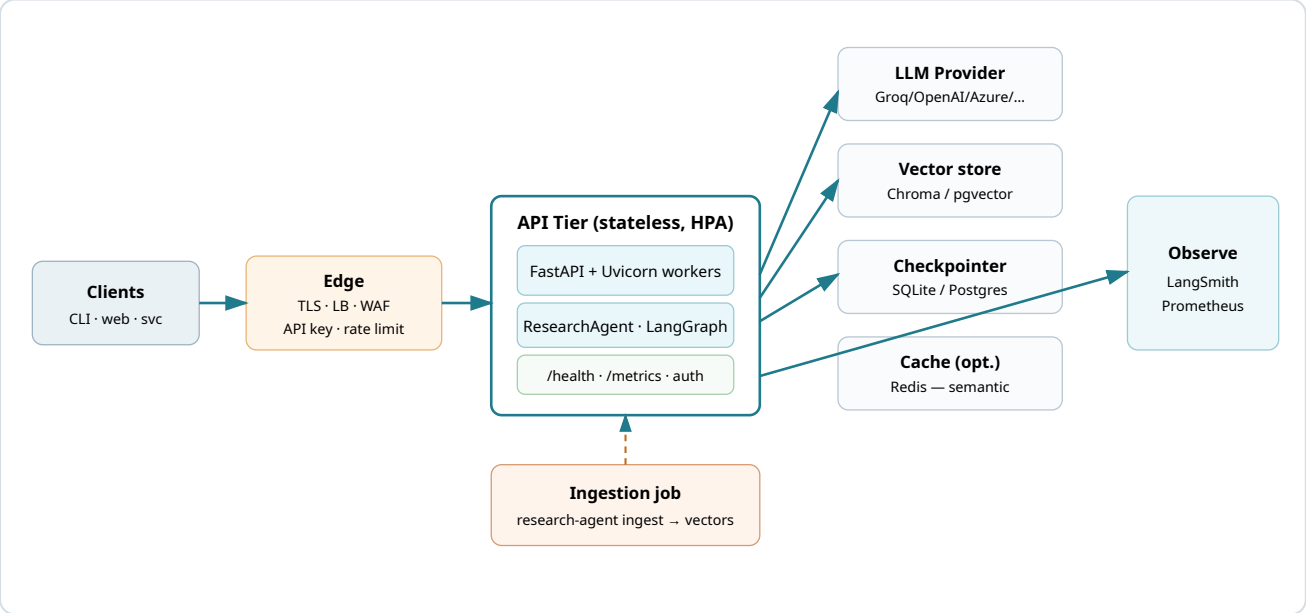


Figure 1 — Target runtime. The dashed line is the out-of-band ingestion pipeline that only writes vectors.

The agent core (LangGraph ReAct loop)

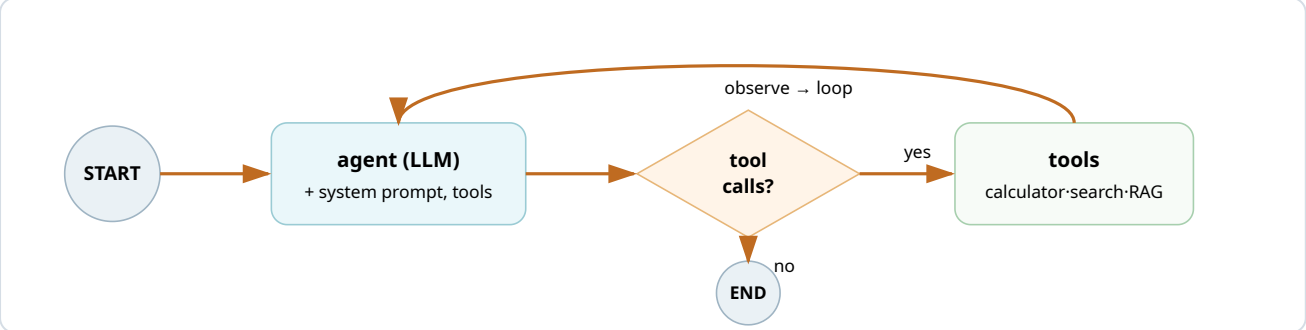
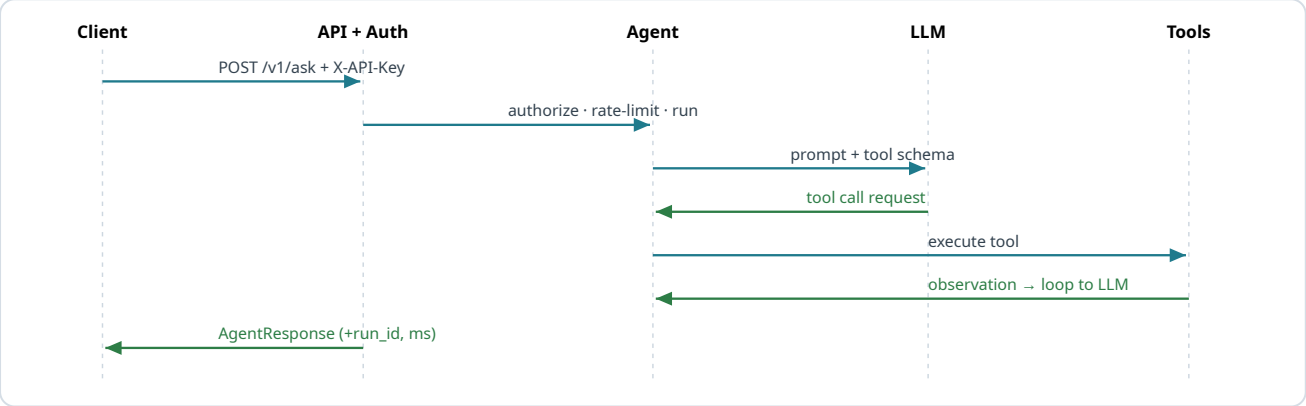


Figure 2 — The agent node calls the model; if it requested tools, the tool node runs them and loops back, bounded by `RECURSION_LIMIT`.

3. Request Lifecycle



Per request: validate & size-check input → authenticate & rate-limit → stamp `run_id` + start timer → run the ReAct loop (LLM ⇄ tools) → normalize the answer to clean text → record metrics → return a typed `AgentResponse`. Any failure becomes an `AgentExecutionError` mapped to HTTP 502; a stuck request is cut off by the timeout middleware (504).

4. Technology Stack & Project Layout

Concern	Technology
Orchestration	LangGraph 1.x (explicit ReAct `StateGraph`)
LLM abstraction	LangChain Core 1.x (`BaseChatModel`)
API	FastAPI + Uvicorn/Gunicorn (ASGI)
State	LangGraph checkpointer — Memory / SQLite / Postgres
RAG	Chroma vector store + local sentence-transformers embeddings
Config	pydantic-settings (typed, validated, env-driven)
CLI	Typer + Rich
Metrics	prometheus-client
Packaging	uv + hatchling (src layout, `py.typed`)
Quality	pytest · ruff · mypy

```
src/agentic_research_agent/
├── api/           # FastAPI app, routes, auth + rate limit, metrics, server entrypoint
│   ├── app.py     # lifespan (build agent once), routes, middleware, error handlers
│   ├── dependencies.py   # API-key auth, sliding-window rate limiter, agent accessor
│   ├── metrics.py        # Prometheus counters/histogram
│   └── server.py         # `research-agent-api` (uvicorn) entrypoint
├── agents/        # graph (ReAct), state, prompts, service facade, checkpointer factory
├── core/          # LLM factory, logging (rich/JSON), observability (LangSmith), except
├── tools/         # calculator · web_search · knowledge_base (RAG)
├── config/        # Settings (all env vars), provider + checkpointer enums
├── schemas/       # AgentRequest / AgentResponse / ToolCallRecord
└── cli.py         # `research-agent` (ask · chat · ingest)
Dockerfile · docker-compose.yml · .dockerignore · .github/workflows/ci.yml
data/knowledge_base/  (RAG corpus)   ·   data/vector_store/ (generated, gitignored)
```

5. LLM Providers

The provider is selected by `LLM_PROVIDER`; the only place a concrete SDK is chosen is the factory in `core/llm.py`. Leave `LLM_MODEL` unset to use the per-provider default.

Provider	`LLM_PROVIDER`	Default model	Required credentials
Groq	`groq`	`llama-3.3-70b-versatile`	`GROQ_API_KEY`
OpenAI	`openai`	`gpt-4o-mini`	`OPENAI_API_KEY`
Azure OpenAI	`azure`	(deployment name)	`AZURE_OPENAI_API_KEY`, `AZURE_OPENAI_ENDPOINT`, `AZURE_OPENAI_DEPLOYMENT`, `AZURE_OPENAI_API_VERSION`
Anthropic	`anthropic`	`claude-sonnet-4-6`	`ANTHROPIC_API_KEY`
Gemini	`gemini`	`gemini-2.5-flash`	`GOOGLE_API_KEY` or `GEMINI_API_KEY`
Ollama (local)	`ollama`	`llama3.1`	`OLLAMA_BASE_URL`

> **Azure note:** Azure routes by *deployment name*, not a model name — set `AZURE_OPENAI_DEPLOYMENT` and leave `LLM_MODEL` unset. The endpoint looks like `https://<resource>.openai.azure.com/`. Do not put trailing commas/quotes on any `.env` value — they become part of the value and cause 404s.

6. Configuration Reference

All settings come from environment variables (or a local `.env`) and are validated on load. Copy `.env.example` to `.env` to start.

Variable	Default	Purpose
`ENVIRONMENT`	`development`	`production` → JSON logs & open-API warning
`LOG_LEVEL`	`INFO`	DEBUG…CRITICAL
`LLM_PROVIDER`	`groq`	groq·openai·azure·anthropic·gemini·ollama
`LLM_MODEL`	provider default	Optional override
`LLM_TEMPERATURE` / `LLM_MAX_TOKENS`	`0.1` / `1024`	Inference params
`LLM_TIMEOUT_SECONDS` / `LLM_MAX_RETRIES`	`60` / `2`	Per-call timeout & transient-error retries
`CHECKPOINTER`	`memory`	memory·sqlite·postgres
`SQLITE_PATH` / `POSTGRES_DSN`	—	State backend location
`EMBEDDING_MODEL`	`all-MiniLM-L6-v2`	Local embeddings
`CHUNK_SIZE` / `CHUNK_OVERLAP` / `RETRIEVER_TOP_K`	`1000` / `150` / `4`	RAG tuning
`API_HOST` / `API_PORT` / `API_WORKERS`	`0.0.0.0` / `8000` / `1`	Server bind & workers
`API_KEYS`	—	Comma-separated keys; empty = auth disabled
`CORS_ORIGINS`	`*`	Allowed origins

Variable	Default	Purpose
`RATE_LIMIT_ENABLED` / `RATE_LIMIT_PER_MINUTE`	`true` / `60`	Per-key/IP budget
`REQUEST_TIMEOUT_SECONDS` / `MAX_QUESTION_CHARS`	`120` / `4000`	Edge timeout & input cap
`LANGSMITH_TRACING` / `LANGSMITH_API_KEY`	`false` / —	Distributed tracing

7. Run Locally — Step by Step

Step 1 — Prerequisites

Python 3.13+ and uv. Verify:

```
python --version    # 3.13+
uv --version
```

Step 2 — Install dependencies

```
uv sync                    # runtime + dev deps into .venv
# (optional) Postgres checkpointer support:
uv sync --extra postgres
```

Step 3 — Configure

```
cp .env.example .env
```

Edit `.env` and set **one** provider + its key. Examples:

```
# Groq (free)
LLM_PROVIDER=groq
GROQ_API_KEY=...

# Azure OpenAI
LLM_PROVIDER=azure
AZURE_OPENAI_API_KEY=...
AZURE_OPENAI_ENDPOINT=https://<resource>.openai.azure.com/
AZURE_OPENAI_DEPLOYMENT=<deployment-name>
AZURE_OPENAI_API_VERSION=2024-10-21

# Gemini
LLM_PROVIDER=gemini
GEMINI_API_KEY=...
```

> **Do not** add trailing commas/quotes to `.env` values — they are read verbatim and cause auth/404 errors.

Step 4 — Build the knowledge base (RAG)

First run downloads the embedding model (~80 MB) once, then indexes `data/knowledge_base/`.

```
uv run research-agent ingest          # add --force to rebuild after edits
```

Step 5 — Use the CLI

```
uv run research-agent ask "What is the ReAct pattern? And what is 23*19?"
uv run research-agent chat                      # interactive, with memory
uv run research-agent chat "start with this question"   # optional seed
uv run research-agent ask "..." --thread session-1      # separate memory
```

Step 6 — Run the API locally

```
uv run research-agent-api          # http://localhost:8000  (Swagger UI at /docs)
```

Smoke test in another terminal:

```
curl localhost:8000/health/ready
curl -X POST localhost:8000/v1/ask \
  -H "Content-Type: application/json" -H "X-API-Key: $API_KEYS" \
  -d '{"question":"What is 23*19?","thread_id":"demo"}'
```

Step 7 — Validate (before committing/deploying)

```
uv run pytest          # 64 tests, offline
uv run ruff check . && uv run ruff format --check .
uv run mypy
```

8. The HTTP API

Endpoint	Method	Auth	Description
`/v1/ask`	POST	API key + rate limit	Answer a question → `AgentResponse` (answer, run_id, duration_ms, tool_calls)
`/v1/stream`	POST	API key + rate limit	Stream the run as Server-Sent Events
`/health/live`	GET	none	Liveness (process up)
`/health/ready`	GET	none	Readiness (vector store + LLM reachable) → 200/503
`/metrics`	GET	none	Prometheus metrics
`/docs`	GET	none	Swagger UI

Every response carries an `X-Request-ID`; requests exceeding `REQUEST_TIMEOUT_SECONDS` return 504; over-long inputs return 413; missing/invalid key returns 401; over-budget callers get 429.

9. Deploy to a Server — Step by Step

Option A — Docker (single container)

A1. Build the image

```
docker build -t agentic-research-agent:0.1.0 .
```

The multi-stage build installs deps with uv, runs as a non-root user, pre-warms the embedding model, and defines a `/health/ready` HEALTHCHECK.

A2. Run it

```
docker run -d --name research-agent -p 8000:8000 \
  --env-file .env \
  -e ENVIRONMENT=production -e CHECKPOINTER=sqlite \
  -e API_KEYS="$STRONG_KEY" \
  -v ra_state:/app/data \
  agentic-research-agent:0.1.0
```

The container starts gunicorn with uvicorn workers. Secrets come from the runtime environment — never baked into the image.

Option B — Docker Compose (API + Postgres)

B1. Bring up the stack

```
cp .env.example .env      # set provider key + API_KEYS
docker compose up --build
```

Compose runs the API with a durable **Postgres** checkpointer (`CHECKPOINTER=postgres`) using the `pgvector` Postgres image, with health-gated startup. Good for a single-host always-on service.

Option C — Kubernetes (scalable)

C1. Push the image & create secrets

```
docker tag agentic-research-agent:0.1.0 $REGISTRY/agentic-research-agent:0.1.0
docker push $REGISTRY/agentic-research-agent:0.1.0
```

```
kubectl create secret generic research-agent-secrets \
  --from-literal=GROQ_API_KEY=... --from-literal=API_KEYS=... \
  --from-literal=POSTGRES_DSN=postgresql://user:pass@pg:5432/agent
```

C2. Deployment + probes + autoscaling

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: research-agent }
spec:
  replicas: 3
  selector: { matchLabels: { app: research-agent } }
  template:
    metadata: { labels: { app: research-agent } }
    spec:
      containers:
        - name: api
          image: REGISTRY/agentic-research-agent:0.1.0
          ports: [{ containerPort: 8000 }]
          env:
            - { name: ENVIRONMENT, value: production }
            - { name: CHECKPOINTER, value: postgres }
          envFrom: [{ secretRef: { name: research-agent-secrets } }]
          readinessProbe: { httpGet: { path: /health/ready, port: 8000 }, initialDela
          livenessProbe:  { httpGet: { path: /health/live,  port: 8000 }, periodSecon
          resources:
            requests: { cpu: "500m", memory: "1Gi" }
            limits:   { cpu: "2",    memory: "2Gi" }
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: research-agent }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: research-agent }
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization, averageUtilization: 65 } }
```

Add a `Service` + `Ingress` (TLS) in front. Readiness gates traffic until the model is loaded and dependencies are reachable; liveness restarts a wedged pod.

Option D — Serverless container (Cloud Run / ECS Fargate / Container Apps)

D1. Deploy the image

Push the image and let the platform handle TLS, autoscaling, and rollout. Set `min-instances ≥ 1` (the embedding-model load makes cold starts heavy) and a generous startup/health timeout. Inject secrets from the platform's secret manager.

```
# Cloud Run example
gcloud run deploy research-agent \
  --image $REGISTRY/agentic-research-agent:0.1.0 \
  --min-instances 1 --max-instances 20 --concurrency 20 \
  --cpu 2 --memory 2Gi --port 8000 \
  --set-secrets API_KEYS=research-api-keys:latest,GROQ_API_KEY=groq-key:latest
```

Tuning workers: set gunicorn workers to roughly `2 × vCPU` for this I/O-bound workload (each request mostly waits on the LLM). Override the image command or set `API_WORKERS` when using `research-agent-api`.

10. Scaling Strategies

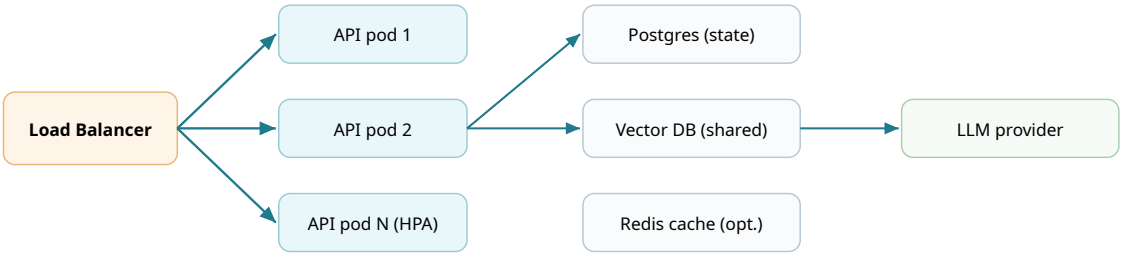


Figure 3 — Scale the stateless API tier horizontally; keep all state in shared backends so any pod can serve any conversation.

How to scale on servers

Dimension	Bottleneck	What to do
Throughput	Concurrent requests; LLM latency dominates each	**Scale out** replicas (K8s HPA / serverless autoscaling) on CPU + concurrency. Within a pod, run multiple uvicorn workers (~2×vCPU).
State	In-memory state can't be shared	Use `CHECKPOINTER=postgres` so every replica reads the same threads; add **PgBouncer** connection pooling.
Vector search	Local Chroma file is node-bound	Move to a **shared/served vector DB** (pgvector, Qdrant, managed) with proper HNSW indexing & replicas.
Embeddings	CPU-bound local model under load	Run embeddings as a dedicated service or use hosted embeddings; GPU nodes for large ingestion. Keep ingestion off the serving path.
Provider limits	LLM rate limits / quotas (429)	Request quota increases; add **multi-provider fallback** (config swap); queue + backpressure; cache repeated answers.
Cost & latency	Token spend, repeated questions	**Semantic + prompt caching**; model routing/cascade (cheap model first); per-tenant

Dimension	Bottleneck	What to do
		token budgets; stream tokens for perceived latency.
Cold starts	Embedding model load at boot	Bake model into the image (done); keep `min-instances ≥ 1` / a warm pool; generous readiness delay.

> **Golden rule:** the API tier is stateless by design, so scaling is "add replicas." The real work is making sure *state* (memory, vectors, cache) lives in shared backends and that you respect the LLM provider's rate limits.

Capacity sizing (rule of thumb)

- Each request is mostly idle waiting on the LLM, so workers can exceed core count. Start at `workers = 2 × vCPU`, then tune by watching CPU and p95 latency.
- Estimate replicas: `peak_RPS × avg_seconds_per_request ÷ (workers × target_utilization)`. Example: 20 RPS × 4 s ÷ (4 × 0.65) ≈ 31 concurrent → ~8 pods at 4 workers.
- Set HPA `maxReplicas` below your LLM provider's concurrency/quota ceiling — otherwise you scale into 429s.

11. Observability

Health

`/health/live` (restart signal) and `/health/ready` (traffic gate) — wired to K8s probes and the container HEALTHCHECK.

Metrics

`/metrics` exposes request counts, run duration histogram, tool-call counts, and run outcomes for Prometheus + Grafana.

Logs

Rich console in dev, structured **JSON** in production (`ENVIRONMENT=production`), with `run_id`, `thread_id`, `duration_ms`, tool counts.

Tracing

Set `LANGSMITH_TRACING=true` + `LANGSMITH_API_KEY` for span-level traces of every LLM and tool call.

Suggested SLOs & alerts

- Availability ≥ 99.5% non-5xx on `/v1/ask`; p95 latency under target; run success > 98%.
- Alert on: 401/429 spikes, rising `agent_run_failed`, readiness failures, token/cost anomalies, climbing empty-retrieval rates.

12. CI/CD

The included GitHub Actions workflow runs on every push/PR:

```
uv sync --frozen --extra postgres
uv run ruff check .        # lint
uv run ruff format --check .
uv run mypy                # types
uv run pytest -q           # 64 tests (offline)
docker build .             # image builds
```

For releases, extend with image scanning (e.g. Trivy), a registry push on tags, and a progressive rollout (canary → 100%) with automatic rollback on SLO breach. For LLM apps, add an **offline eval gate** (answer quality / retrieval relevance) that must pass before promotion.

13. Security

- Secrets injected at runtime from a managed store; `.env` is git-ignored; keys held as `SecretStr`. Never bake secrets into images.
- TLS + WAF + API-key auth at the edge; per-key rate limits and quotas.
- Calculator uses an AST allow-list (no `eval`); input length capped; treat retrieved/web content as untrusted (prompt-injection awareness).
- Run as non-root in the container; scan images and pin `uv.lock`; restrict egress for outbound LLM/search calls; keep audit logs (run_id, inputs, tool calls).

14. Troubleshooting

Symptom	Cause	Fix
`404 model_not_found` (OpenAI)	Provider switched but `LLM_MODEL` still set to another provider's model	Unset `LLM_MODEL` (use provider default) or set a valid model for the provider
`404 Resource not found` (Azure)	Wrong endpoint/deployment — often a stray trailing comma/space in `.env`	Clean `AZURE_OPENAI_*` values (no trailing comma/quote); verify deployment name & api-version
`429 insufficient_quota / rate limit`	Provider account quota or per-minute/day limit reached	Add billing / raise quota, wait for reset, or switch provider; `LLM_MAX_RETRIES` covers only transient 429s
Run aborts on tool-heavy prompts (intermittent)	Provider returned a malformed `tool_use_failed`	Handled: the agent parses known failure formats and resamples; no action needed
Answer shows raw `[{'type':'text',…}]`	Provider returns content blocks	Handled: responses are normalized to clean text
Memory lost after restart	`CHECKPOINTER=memory`	Use `sqlite` (single node) or `postgres` (multi-replica)
`401` from the API	Missing/invalid `X-API-Key`	Send a key listed in `API_KEYS`
Slow first request	Embedding model downloading/loading	Pre-warm (image bakes it); keep a warm instance